

RAPPORT : Projet de gestion de plateforme universitaire (SGBD)

1. Introduction

Notre projet repose sur la modélisation d'une base de données destinée à une plateforme universitaire. Dans cette optique, nous avons analysé, structuré et organisé l'ensemble des informations nécessaires à son fonctionnement. Pour mettre en place cette organisation, nous avons adopté la méthode Entité-Relation (ER), largement reconnue pour sa capacité à représenter les données de manière claire, cohérente et structurée.

L'objectif principal est d'élaborer un Modèle Conceptuel de Données (MCD), qui servira de fondation à la modélisation de la base de données relationnelle. Ce modèle permet non seulement de structurer efficacement les informations, mais aussi d'en faciliter l'exploitation ultérieure et d'assurer une gestion optimale des données au sein de la plateforme.

2. Analyse et identification des besoins

A) Objectif du projet

L'objectif de notre projet est de concevoir une base de données capable de :

- **Gérer et centraliser** l'ensemble des informations relatives aux étudiants, aux cours, aux enseignants et aux examens
- **Faciliter la gestion du parcours universitaire**, notamment le suivi du cursus, les inscriptions et la planification des cours
- **Assurer un suivi précis des résultats des étudiants**, en regroupant les examens et les notes associées
- **Optimiser la planification des ressources**, telles que les salles, les horaires ou encore la disponibilité des enseignants

B) Entités principales identifiées

Les entités principales sont :

- **Etudiant** → Représente les personnes inscrites dans la faculté et suivant un cursus
- **Enseignant** → Représente les personnes donnant les cours et supervisant les examens
- **Cours** → Représente les matières enseignées, liées à un cursus et assurées par un enseignant
- **Inscription** → Représente l'enregistrement d'un étudiant à un cours pour un semestre donné
- **Examen** → Représente une session d'évaluation associée à un cours et un semestre
- **Note** → Résultats obtenus par un étudiant à un examen
- **Semestre** → Représente les périodes de l'année universitaire
- **Cursus** → Représente un parcours d'étude regroupant des cours spécifiques
- **Séance_Cours** → Représente une séance de cours programmée avec une date, un horaire et une salle
- **Salle** → Représente un lieu physique dans lequel se déroulent les séances de cours et les examens

3. **Modélisation Conceptuelle du projet**

A) Entités et leurs attributs

- ❖ **Étudiant** : num_etudiant, nom, prenom, date_naiss, email, id_cursus
- ❖ **Enseignant** : id_ens, nom, prenom, domaine
- ❖ **Cours** : id_cours, nom_cours, id_ens, id_cursus
- ❖ **Inscription** : id_inscription, num_etudiant, id_cours, date_inscription, statut
- ❖ **Examen** : id_exam, id_cours, id_semestre, horaire_deb, horaire_fin

- ❖ **Note** : num_etudiant, id_exam, note
- ❖ **Semestre** : id_semestre, nom_semestre
- ❖ **Cursus** : id_cursus, nom_cursus
- ❖ **Séance_Cours** : id_seance, id_cours, numero_salle, horaire_deb, horaire_fin
- ❖ **Salle** : numero_salle, capacite, horaire_ouverture, horaire_fermeture

Rq : Souligné = PK // Surligné = FK

B) Relations et Cardinalités

- **Inscrit / Concerne** (Étudiant - Inscription - Cours)
 - Cardinalité (1,N) : Un étudiant s'inscrit à plusieurs cours
 - Cardinalité (1,N) : Un cours concerne plusieurs étudiants
- **Appartient** (Cursus - Cours)
 - Cardinalité (1,N) : Un cursus appartient à plusieurs cours
- **Dispense** (Enseignant - Cours)
 - Cardinalité (1,N) : Un enseignant dispense un ou plusieurs cours
- **Évalue** (Cours - Examen)
 - Cardinalité (1,N) : Un cours peut inclure plusieurs examen et chaque examen est lié à un seul cours
- **Se déroule** (Semestre - Examen)
 - Cardinalité (1,N) : Un semestre comprend plusieurs examens
- **Évalue** (Examen - Notes)
 - Cardinalité (1,N) : Un examen évalue plusieurs étudiants, plusieurs notes sont attribuées à un examen
- **Obtient** (Étudiant - Notes)
 - Cardinalité (1,N) : Un étudiant obtient plusieurs notes durant l'année
- **Accueille** (Salle - Séance_Cours)
 - Cardinalité (1,N) : Une salle accueille plusieurs séances de cours
- **Concerne** (Cours - Séance_Cours)
 - Cardinalité (1,N) : Un cours donne lieu à plusieurs séances de cours durant le semestre

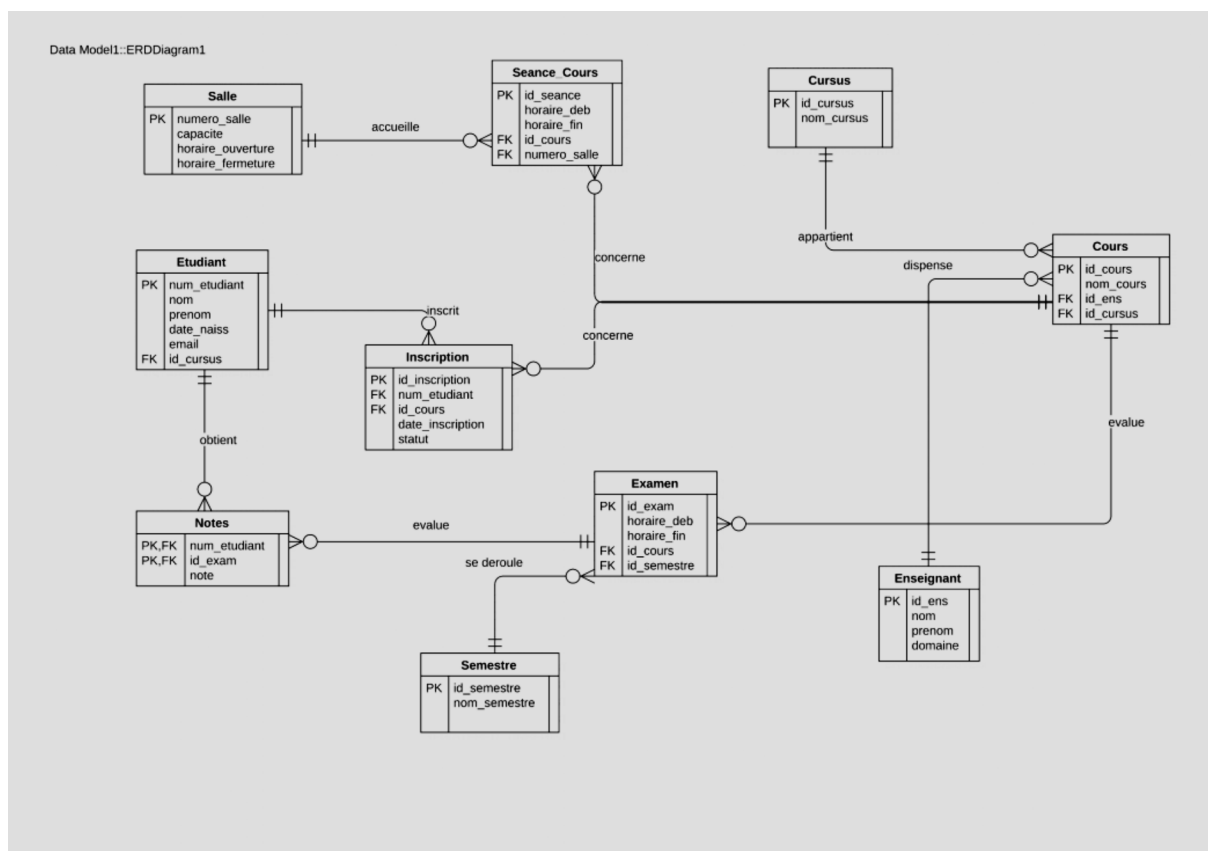
C) Justification des clés

- **Les clés primaires (PK)** : garantie l'unicité de chaque entité
- **Les clés étrangères (FK)** : permet de relier les entités entre-elles, assurant l'intégrité des relations

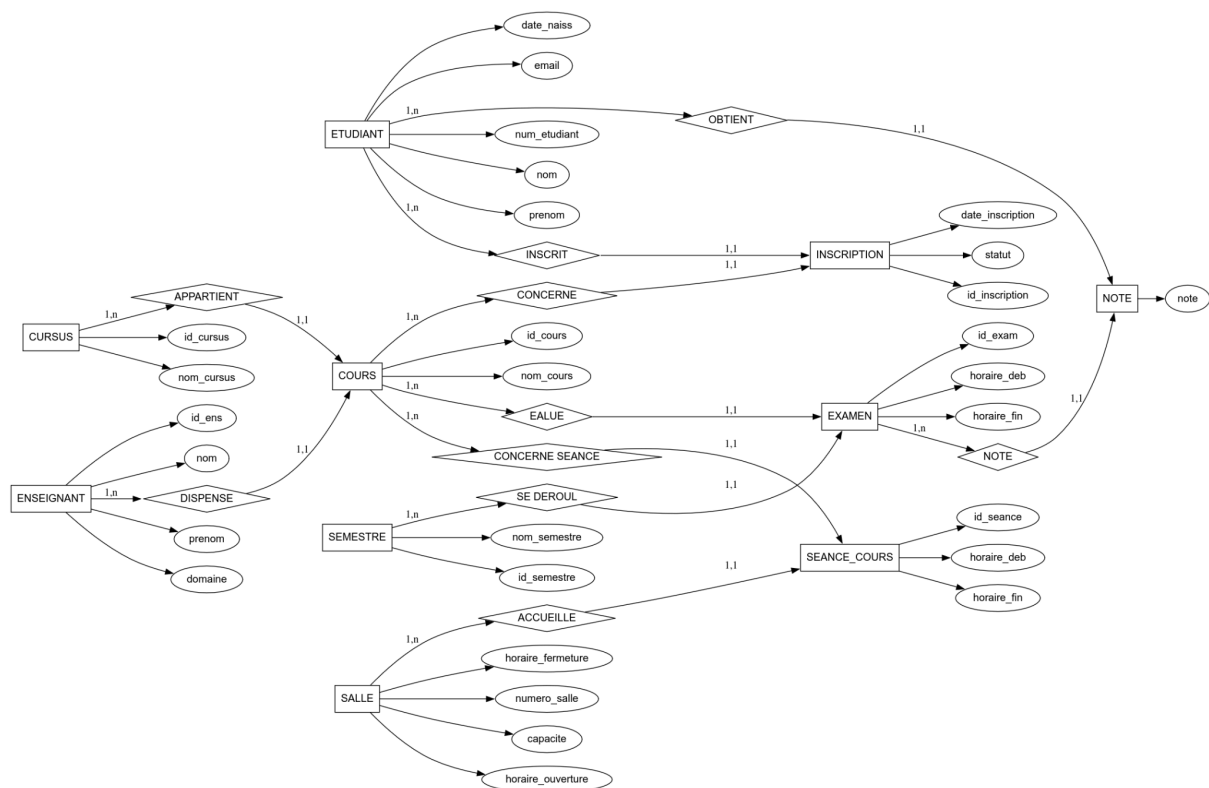
3. Diagramme Entité-Relation

Dans le but de structurer les informations et de comprendre l'organisation des données, nous avons conçu deux diagrammes Entité-Relation (ER). La réalisation de ces ER s'est appuyée sur un Modèle Conceptuel des Données (MCD), construit selon la méthode Entité-Relation. Celui-ci nous a permis d'identifier les entités clés du système, de définir leurs attributs, ainsi que de formaliser les relations qui existent entre elles. Les diagrammes obtenus synthétisent de manière visuelle et cohérente l'ensemble des interactions au sein de notre base de données.

Nous avons d'abord réalisé un diagramme ER en notation UML afin de structurer les entités et mieux comprendre les interactions du système.



Dans un second temps, nous avons produit le diagramme Entité-Relation conforme à la méthode MERISE, avec les notations standard : entités en rectangles, attributs en ovales et associations en losanges. Toutes les relations du modèle présentent des cardinalités de type (1,N), conformément aux règles du domaine étudié.



4. Améliorations et Limitations

A) Améliorations possibles

☐ Gérer la disponibilité des enseignants et des salles

→ Ajouter une vérification automatique ou une table "horaires"

☐ Ajouter les absences et la présence aux examens

→ Ajouter une Table Absence(num_etudiant, id_seance)

☐ Intégrer la gestion des sessions d'examens (rattrapage, session 2)

→ Ajouter session_exam (normale / rattrapage)

B) Limitations

Bien que notre modèle réponde aux besoins essentiels de la plateforme universitaire, certaines limites subsistent. Le système ne prend pas encore en compte la gestion des conflits d'horaires entre les salles, les étudiants ou les enseignants, ce qui peut entraîner des incohérences dans la planification. Le modèle ne gère pas non plus les capacités des salles lors des inscriptions, ni les notions avancées comme les contrôles continus, les coefficients ou les groupes d'étudiants. Enfin, aucune vérification automatique n'est prévue pour éviter les suppressions incohérentes dans les données. Malgré ces limites, la base reste pleinement fonctionnelle pour un usage académique simplifié.

5. Conclusion

Pour mener à bien ce projet, nous avons suivi les consignes du professeur tout en nous appuyant sur les quarante-et-une requêtes fournies en exemple. Ces exigences nous ont guidés dans la construction de notre Modèle Conceptuel de Données (MCD) ainsi que de notre diagramme Entité-Relation (ER), et ont directement influencé la structure finale de notre base de données. Ce travail nous a permis de nous confronter à une situation proche de celle rencontrée dans un projet d'entreprise, où il est nécessaire d'analyser des besoins réels et de proposer une modélisation cohérente et exploitable. Nous sommes satisfaits du résultat obtenu, qui reflète une base de données claire, fonctionnelle et conforme aux objectifs fixés.

6. Base Des Données et Requêtes

A) Base des Données

Ci-joint notre base des données générée à partir de nos deux diagrammes ER ↓

```
-- =====  
-- Création des tables  
-- =====
```

```
CREATE TABLE Coursus (  
    id_cursus INT PRIMARY KEY,  
    nom_cursus VARCHAR(100)  
);
```

```
CREATE TABLE Enseignant (  
    id_ens INT PRIMARY KEY,
```

```
    nom VARCHAR(100),  
    prenom VARCHAR(100),  
    domaine VARCHAR(100)  
);
```

```
CREATE TABLE Etudiant (  
    num_etudiant INT PRIMARY KEY,  
    nom VARCHAR(100),  
    prenom VARCHAR(100),  
    date_naiss DATE,  
    email VARCHAR(150),  
    id_cursus INT,  
    FOREIGN KEY (id_cursus) REFERENCES Coursus(id_cursus)  
);
```

```
CREATE TABLE Cours (  
    id_cours INT PRIMARY KEY,  
    nom_cours VARCHAR(100),  
    id_ens INT,  
    id_cursus INT,  
    FOREIGN KEY (id_ens) REFERENCES Enseignant(id_ens),  
    FOREIGN KEY (id_cursus) REFERENCES Coursus(id_cursus)  
);
```

```
CREATE TABLE Inscription (  
    id_inscription INT PRIMARY KEY,  
    num_etudiant INT,  
    id_cours INT,  
    date_inscription DATE,  
    statut VARCHAR(20) DEFAULT 'active',  
    FOREIGN KEY (num_etudiant) REFERENCES Etudiant(num_etudiant),  
    FOREIGN KEY (id_cours) REFERENCES Cours(id_cours)  
);
```

```
CREATE TABLE Semestre (  
    id_semestre INT PRIMARY KEY,  
    nom_semestre VARCHAR(50)  
);
```

```
CREATE TABLE Examen (  
    id_exam INT PRIMARY KEY,  
    id_cours INT,  
    id_semestre INT,  
    horaire_deb DATETIME,  
    horaire_fin DATETIME,  
    FOREIGN KEY (id_cours) REFERENCES Cours(id_cours),  
    FOREIGN KEY (id_semestre) REFERENCES Semestre(id_semestre)
```

);

```
CREATE TABLE Notes (  
    num_etudiant INT,  
    id_exam INT,  
    note FLOAT,  
    PRIMARY KEY (num_etudiant, id_exam),  
    FOREIGN KEY (num_etudiant) REFERENCES Etudiant(num_etudiant),  
    FOREIGN KEY (id_exam) REFERENCES Examen(id_exam)  
);
```

```
CREATE TABLE Salle (  
    numero_salle INT PRIMARY KEY,  
    capacite INT,  
    horaire_ouverture TIME,  
    horaire_fermeture TIME  
);
```

```
CREATE TABLE Seance_Cours (  
    id_seance INT PRIMARY KEY,  
    id_cours INT,  
    numero_salle INT,  
    horaire_deb DATETIME,  
    horaire_fin DATETIME,  
    FOREIGN KEY (id_cours) REFERENCES Cours(id_cours),  
    FOREIGN KEY (numero_salle) REFERENCES Salle(numero_salle)  
);
```

```
-- =====  
-- Insertion des données  
-- =====
```

```
INSERT INTO Cursus (id_cursus, nom_cursus) VALUES  
(1, 'Informatique'),  
(2, 'Droit'),  
(3, 'Staps'),  
(4, 'Musique');
```

```
INSERT INTO Enseignant (id_ens, nom, prenom, domaine) VALUES  
(1, 'Dupont', 'Frederic', 'Informatique'),  
(2, 'Emengha', 'Aksel', 'Droit'),  
(3, 'Poulet', 'Sarah', 'Musique'),  
(4, 'Bodin', 'Lucie', 'Staps'),  
(5, 'Martin', 'Claire', 'Histoire');
```

```
INSERT INTO Etudiant (num_etudiant, nom, prenom, date_naiss, email, id_cursus) VALUES  
(1, 'Le Cloirec', 'Emma', '2006-02-14', 'emma.lecloirec@example.com', 1),  
(2, 'Bouzerara', 'Younes', '2001-11-03', 'younes.bouzerara@example.com', 2),
```



```
(3, 'Lecomte', 'Alice', '1998-02-20', 'alice.lecomte@example.com', 1),
(4, 'Gomez', 'Jules', '1990-08-10', 'jules.gomez@example.com', 3),
(5, 'Capron', 'Antoine', '2003-03-22', 'antoine.capron@example.com', 4);
```

```
INSERT INTO Cours (id_cours, nom_cours, id_ens, id_cursus) VALUES
(1, 'Programmation', 1, 1),
(2, 'Base des Données', 1, 1),
(3, 'Droit de la Famille', 2, 2),
(4, 'Droit Administratif', 2, 2),
(5, 'Solfege', 3, 4),
(6, 'Althetisme', 4, 3);
```

```
INSERT INTO Semestre (id_semestre, nom_semestre) VALUES
(1, 'Semestre 1'),
(2, 'Semestre 2');
```

```
INSERT INTO Inscription (id_inscription, num_etudiant, id_cours, date_inscription, statut)
VALUES
(1, 1, 1, '2025-07-10', 'active'),
(2, 1, 2, '2025-07-12', 'active'),
(3, 2, 3, '2025-08-01', 'active'),
(4, 3, 1, '2025-08-15', 'annulee'),
(5, 3, 4, '2025-09-01', 'active'),
(6, 4, 1, '2025-09-05', 'active'),
(7, 5, 5, '2025-09-10', 'annulee');
```

```
INSERT INTO Examen (id_exam, id_cours, id_semestre, horaire_deb, horaire_fin) VALUES
(1, 1, 1, '2025-02-20 10:00', '2025-02-20 12:00'),
(2, 1, 2, '2025-06-15 09:00', '2025-06-15 11:00'),
(3, 3, 1, '2025-01-25 14:00', '2025-01-25 16:00'),
(4, 5, 2, '2025-06-18 13:00', '2025-06-18 15:00');
```

```
INSERT INTO Notes (num_etudiant, id_exam, note) VALUES
(1, 1, 12),
(2, 3, 14),
(1, 2, 16),
(3, 1, 9),
(5, 4, 18);
```

```
INSERT INTO Salle (numero_salle, capacite, horaire_ouverture, horaire_fermeture)
VALUES
(101, 40, '08:00', '18:00'),
(102, 50, '08:00', '20:00');
```

```
INSERT INTO Seance_Cours (id_seance, id_cours, numero_salle, horaire_deb, horaire_fin)
VALUES
(1, 1, 101, '2025-02-10 09:00', '2025-02-10 11:00'),
```

(2, 3, 102, '2025-04-05 14:00', '2025-04-05 16:00'),
(3, 5, 101, '2025-03-20 13:00', '2025-03-20 15:00');

B) Requêtes

Dans le cadre du suivi de notre projet, un fichier contenant 41 requêtes nous a été fourni à titre d'exemple. Nous les avons rédigées et les voici :

-- 1. Afficher les étudiants qui n'ont pas encore passé d'examen

```
SELECT e.*  
FROM Etudiant e  
LEFT JOIN Notes n ON e.num_etudiant = n.num_etudiant  
WHERE n.id_exam IS NULL;
```

-- 2. Nombre total d'étudiants inscrits

```
SELECT COUNT(*) AS total_etudiants  
FROM Etudiant;
```

-- 3. Étudiants ayant amélioré leurs notes entre 2 semestres

```
SELECT n1.num_etudiant  
FROM Notes n1  
JOIN Notes n2 ON n1.num_etudiant = n2.num_etudiant  
JOIN Examen e1 ON n1.id_exam = e1.id_exam  
JOIN Examen e2 ON n2.id_exam = e2.id_exam  
WHERE e1.id_semestre = 1 AND e2.id_semestre = 2  
GROUP BY n1.num_etudiant  
HAVING AVG(n2.note) > AVG(n1.note);
```

-- 4. Moyenne des notes par cours

```
SELECT c.nom_cours, AVG(n.note) AS moyenne  
FROM Notes n  
JOIN Examen ex ON n.id_exam = ex.id_exam  
JOIN Cours c ON ex.id_cours = c.id_cours  
GROUP BY c.nom_cours;
```

-- 5. Détail des enseignants qui encadrent des examens

```
SELECT DISTINCT e.nom, e.prenom, ex.id_exam, c.nom_cours  
FROM Enseignant e  
JOIN Cours c ON c.id_ens = e.id_ens  
JOIN Examen ex ON ex.id_cours = c.id_cours;
```

-- 6. Nombre total d'examens par cours

```
SELECT c.nom_cours, COUNT(ex.id_exam) AS nb_examens  
FROM Cours c  
LEFT JOIN Examen ex ON ex.id_cours = c.id_cours
```

GROUP BY c.nom_cours;

-- 7. Répartition des étudiants par tranche d'âge

```
SELECT
CASE
  WHEN TIMESTAMPDIFF(YEAR, date_naiss, CURDATE()) < 20 THEN '-20 ans'
  WHEN TIMESTAMPDIFF(YEAR, date_naiss, CURDATE()) BETWEEN 20 AND 30 THEN
'20-30 ans'
  ELSE 'plus de 30 ans'
END AS tranche_age,
COUNT(*) AS total
FROM Etudiant
GROUP BY tranche_age;
```

-- 8. Étudiants avec moyenne générale > 15

```
SELECT e.num_etudiant, e.nom, e.prenom, AVG(n.note) AS moyenne
FROM Notes n
JOIN Etudiant e ON e.num_etudiant = n.num_etudiant
GROUP BY e.num_etudiant
HAVING AVG(n.note) > 15;
```

-- 9. Enseignants ne dispensant aucun cours

```
SELECT *
FROM Enseignant e
LEFT JOIN Cours c ON e.id_ens = c.id_ens
WHERE c.id_cours IS NULL;
```

-- 10. Cours dispensés par un enseignant donné

```
SELECT c.nom_cours
FROM Cours c
JOIN Enseignant e ON c.id_ens = e.id_ens
WHERE e.nom = 'Lemoine';
```

-- 11. Nombre d'inscriptions par cours (ordre décroissant)

```
SELECT c.nom_cours, COUNT(i.id_inscription) AS nb_inscriptions
FROM Cours c
LEFT JOIN Inscription i ON i.id_cours = c.id_cours
GROUP BY c.nom_cours
ORDER BY nb_inscriptions DESC;
```

-- 12. Étudiants ayant obtenu la meilleure note à chaque examen

```
SELECT n.*
FROM Notes n
WHERE n.note = (
  SELECT MAX(note)
  FROM Notes
  WHERE id_exam = n.id_exam
);
```

-- 13. Inscriptions faites après une date donnée

```
SELECT *  
FROM Inscription  
WHERE date_inscription > '2025-02-01';
```

-- 14. Nombre moyen d'inscriptions par étudiant

```
SELECT AVG(nb) AS moyenne  
FROM (  
    SELECT num_etudiant, COUNT(*) AS nb  
    FROM Inscription  
    GROUP BY num_etudiant  
) t;
```

-- 15. Inscriptions faites pour un étudiant donné

```
SELECT i.*  
FROM Inscription i  
JOIN Etudiant e ON e.num_etudiant = i.num_etudiant  
WHERE e.nom = 'Dupont' AND e.prenom = 'Alice';
```

-- 16. Étudiants inscrits à un cours spécifique

```
SELECT e.*  
FROM Etudiant e  
JOIN Inscription i ON e.num_etudiant = i.num_etudiant  
WHERE i.id_cours = 1;
```

-- 17. Évolution mensuelle des inscriptions

```
SELECT MONTH(date_inscription) AS mois, COUNT(*) AS nb  
FROM Inscription  
WHERE YEAR(date_inscription) = 2025  
GROUP BY mois  
ORDER BY mois;
```

-- 18. Étudiants + moyenne générale par cours

```
SELECT e.num_etudiant, e.nom, e.prenom, c.nom_cours, AVG(n.note) AS moyenne  
FROM Notes n  
JOIN Examen ex ON n.id_exam = ex.id_exam  
JOIN Cours c ON c.id_cours = ex.id_cours  
JOIN Etudiant e ON e.num_etudiant = n.num_etudiant  
GROUP BY e.num_etudiant, c.nom_cours;
```

-- 19. Étudiants inscrits à plus de 3 cours

```
SELECT num_etudiant, COUNT(*) AS nb  
FROM Inscription  
GROUP BY num_etudiant  
HAVING COUNT(*) > 3;
```

-- 20. Enseignants + cours qu'ils dispensent

```
SELECT e.nom, e.prenom, c.nom_cours
FROM Enseignant e
LEFT JOIN Cours c ON c.id_ens = e.id_ens;
```

-- 21. Inscriptions annulées

```
SELECT *
FROM Inscription
WHERE statut = 'annulée';
```

-- 22. Enseignants qui encadrent le plus de cours

```
SELECT e.nom, e.prenom, COUNT(c.id_cours) AS nb
FROM Enseignant e
JOIN Cours c ON c.id_ens = e.id_ens
GROUP BY e.id_ens
ORDER BY nb DESC
LIMIT 1;
```

-- 23. Liste des étudiants (nom, prénom, date de naissance, email)

```
SELECT nom, prenom, date_naiss, email
FROM Etudiant;
```

-- 24. Nombre total d'inscriptions par cours

```
SELECT c.nom_cours, COUNT(i.id_inscription) AS nb
FROM Cours c
LEFT JOIN Inscription i ON i.id_cours = c.id_cours
GROUP BY c.nom_cours;
```

-- 25. Cours avec plus de 50 inscrits

```
SELECT c.nom_cours, COUNT(i.id_inscription) AS nb
FROM Cours c
JOIN Inscription i ON c.id_cours = i.id_cours
GROUP BY c.nom_cours
HAVING COUNT(i.id_inscription) > 50;
```

-- 26. Cours ayant le taux de réussite le plus élevé (note >= 10)

```
SELECT c.nom_cours, SUM(CASE WHEN n.note >= 10 THEN 1 ELSE 0 END) / COUNT(*)
* 100 AS taux_reussite
FROM Notes n
JOIN Examen ex ON n.id_exam = ex.id_exam
JOIN Cours c ON c.id_cours = ex.id_cours
GROUP BY c.nom_cours
ORDER BY taux_reussite DESC;
```

-- 27. Nombre d'inscriptions annulées par mois

```
SELECT MONTH(date_inscription) AS mois, COUNT(*) AS nb
FROM Inscription
WHERE statut = 'annulée'
GROUP BY mois;
```

-- 28. Étudiants inscrits à tous les cours d'un enseignant donné

```
SELECT i.num_etudiant
FROM Inscription i
JOIN Cours c ON i.id_cours = c.id_cours
WHERE c.id_ens = 2 AND i.statut = 'active'
GROUP BY i.num_etudiant
HAVING COUNT(DISTINCT i.id_cours) = (
  SELECT COUNT(*) FROM Cours WHERE id_ens = 2
);
```

-- 29. Étudiants + cours + enseignant pour chaque inscription

```
SELECT e.nom AS etudiant, c.nom_cours, ens.nom AS enseignant
FROM Inscription i
JOIN Etudiant e ON e.num_etudiant = i.num_etudiant
JOIN Cours c ON c.id_cours = i.id_cours
JOIN Enseignant ens ON ens.id_ens = c.id_ens;
```

-- 30. Cours dont la moyenne des notes < 12

```
SELECT c.nom_cours, AVG(n.note) AS moyenne
FROM Notes n
JOIN Examen ex ON n.id_exam = ex.id_exam
JOIN Cours c ON ex.id_cours = c.id_cours
GROUP BY c.nom_cours
HAVING moyenne < 12;
```

-- 31. Date d'inscription par étudiant et par cours

```
SELECT e.nom, c.nom_cours, i.date_inscription
FROM Inscription i
JOIN Etudiant e ON e.num_etudiant = i.num_etudiant
JOIN Cours c ON c.id_cours = i.id_cours;
```

-- 32. Étudiants ayant passé un examen spécifique

```
SELECT DISTINCT e.*
FROM Notes n
JOIN Etudiant e ON e.num_etudiant = n.num_etudiant
WHERE n.id_exam = 1;
```

-- 33. Notes d'un cours donné

```
SELECT e.nom, n.note
FROM Notes n
JOIN Etudiant e ON e.num_etudiant = n.num_etudiant
JOIN Examen ex ON ex.id_exam = n.id_exam
WHERE ex.id_cours = 1;
```

-- 34. Examens d'un cours donné

```
SELECT *
FROM Examen
```

WHERE id_cours = 1;

-- 35. Tous les cours + enseignant responsable

SELECT c.nom_cours, e.nom, e.prenom
FROM Cours c
JOIN Enseignant e **ON** c.id_ens = e.id_ens;

-- 36. Enseignant + nombre d'étudiants inscrits dans leurs cours

SELECT ens.nom, **COUNT**(**DISTINCT** i.num_etudiant) **AS** nb_etudiants
FROM Enseignant ens
JOIN Cours c **ON** c.id_ens = ens.id_ens
JOIN Inscription i **ON** i.id_cours = c.id_cours
WHERE i.statut = 'activée'
GROUP BY ens.id_ens;

-- 37. Cours sans inscriptions

SELECT c.nom_cours
FROM Cours c
LEFT JOIN Inscription i **ON** c.id_cours = i.id_cours
WHERE i.id_inscription **IS NULL**;

-- 38. Moyenne des notes des étudiants par enseignant

SELECT ens.nom, ens.prenom, **AVG**(n.note) **AS** moyenne
FROM Enseignant ens
JOIN Cours c **ON** c.id_ens = ens.id_ens
JOIN Examen ex **ON** ex.id_cours = c.id_cours
JOIN Notes n **ON** n.id_exam = ex.id_exam
GROUP BY ens.id_ens;

-- 39. Étudiants ayant échoué à un examen donné

SELECT e.*
FROM Notes n
JOIN Etudiant e **ON** n.num_etudiant = e.num_etudiant
WHERE n.id_exam = 1 **AND** n.note < 10;

-- 40. Nombre total de cours par enseignant

SELECT e.nom, e.prenom, **COUNT**(c.id_cours) **AS** nb_cours
FROM Enseignant e
LEFT JOIN Cours c **ON** c.id_ens = e.id_ens
GROUP BY e.id_ens;

-- 41. Cours dispensés par un enseignant donné

SELECT c.nom_cours
FROM Cours c
JOIN Enseignant e **ON** c.id_ens = e.id_ens
WHERE e.nom = 'Bernard';